

EFREI PARIS

Master 1 — Cybersecurite

TP VISUALISATION DE DONNEES

Exercices appliques — Securite & Monitoring reseau

Matplotlib & Seaborn
TP Etudiant — Duree : 2h

Duree
2h – 2h30

Livrables
7 scripts .py + graphiques
.png

Exercices
7 exercices + 1 mini-projet

Exercice	Exercice	Niveau
1	Trafic SSH – courbe temporelle	Debutant
2	Alertes par type – barres	Intermediaire
3	Scatter plot – detection d'anomalie	Intermediaire
4	Camembert – repartition protocoles	Avance
5	Distribution des durees	
6	Heatmap de correlation	
7	Violin + boxplot comparatif	
8	Mini-projet – tableau de bord SOC	

Rappel : toujours utiliser fig, ax = plt.subplots() et appeler plt.savefig() AVANT plt.show(). Sauvegarder chaque graphique sous le nom indique dans l'annonce.

Exercice 1 Trafic SSH — Courbe temporelle (Line Plot)

EXERCICE
N° 1

Trafic SSH — Courbe temporelle

Niveau
Debutant

Contexte : Monitoring SSH

L'equipe SOC surveille les connexions SSH entrantes sur un serveur de production. Vous disposez du nombre de tentatives de connexion SSH par heure sur 24 heures. Votre mission : produire une courbe lisible pour identifier les pics d'activite suspect.

Donnees fournies

```
heures = list(range(24))
# Tentatives de connexion SSH par heure (0h = minuit)
tentatives = [3, 2, 1, 1, 2, 4, 8, 15, 22, 18, 20, 19,
              16, 21, 24, 30, 45, 38, 28, 18, 12, 8, 5, 3]
# Seuil d'alerte operationnel
seuil_alerte = 25
```

Travail demande — Fichier : ex1_ssh_trafic.py

- 1 Tracer la courbe des tentatives SSH en fonction de l'heure avec une ligne bleue, des marqueurs circulaires et une epaisseur de 2.
- 2 Ajouter une ligne horizontale rouge en pointilles representant le seuil d'alerte (25 tentatives). Ajouter une legende.
- 3 Colorer en rouge (alpha=0.15) la zone au-dessus du seuil : utiliser `ax.fill_between()` avec une condition.
- 4 Personnaliser : titre, labels des axes (Heure de la journee / Nb tentatives), grille pointillee, graduation toutes les 2h sur l'axe X.
- 5 Sauvegarder sous `ex1_ssh_trafic.png` (dpi=150) et afficher.

Indice — `ax.fill_between()`

```
ax.fill_between(heures, tentatives, seuil_alerte,
                where=[t > seuil_alerte for t in tentatives], # condition
                booleen
                color='red', alpha=0.15, # rouge transparent
                label='Zone critique')
```

RESULTAT ATTENDU

Courbe bleue avec marqueurs, ligne rouge en pointilles a y=25, zone rouge transparente au-dessus du seuil, graduation de 0 a 23 par pas de 2, grille visible, legende.

BONUS

Ajouter une annotation fleche pointant vers le pic maximum (heure 16 : 45 tentatives) avec `ax.annotate()`. Afficher aussi la valeur numerique dans l'annotation.

Exercice 2 Alertes de securite par type — Barres

EXERCICE N° 2

Alertes par type — Bar Chart double

Niveau
Debutant

Contexte : Tableau de bord SOC

Votre SIEM a genere des alertes sur deux semaines consecutives. Vous devez comparer visuellement la repartition des alertes par type entre Semaine 1 et Semaine 2 pour identifier les types d'incidents en hausse.

Donnees fournies

```
types_alertes = ['Brute Force', 'Port Scan', 'DDoS', 'Phishing', 'Malware', 'Exfil.']
semaine_1     = [42, 28, 15, 33, 19, 8]
semaine_2     = [67, 31, 9, 45, 27, 14]
```

Travail demande — Fichier : ex2_alertes_barres.py

- 1 Créer un graphique en barres groupées (deux barres cote a cote pour chaque type) avec numpy pour decaler les positions.
- 2 Colorier les barres Semaine 1 en bleu steelblue et Semaine 2 en corail (coral). Largeur de barre : 0.35.
- 3 Afficher la valeur numerique au-dessus de chaque barre (ax.text() ou ax.bar_label()).
- 4 Ajouter titre, labels, legende, grille horizontale uniquement (ax.grid(axis='y')).
- 5 Sauvegarder sous ex2_alertes_barres.png.

Indice — barres groupées avec numpy

```
import numpy as np

x      = np.arange(len(types_alertes))
width  = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, semaine_1, width, label='Semaine 1',
               color='steelblue')
bars2 = ax.bar(x + width/2, semaine_2, width, label='Semaine 2', color='coral')

ax.set_xticks(x)
ax.set_xticklabels(types_alertes, rotation=20, ha='right')
```

RESULTAT ATTENDU

6 paires de barres groupées bleues/corail, valeurs affichées au sommet, légende, grille horizontale, labels lisibles (pivotés de 20 degrés).

BONUS

Calculer et afficher le taux d'évolution (%) entre S1 et S2 pour chaque type. Colorer le label en rouge si augmentation > 30%, en vert si diminution.

Exercice 3 Detection d'anomalie — Scatter Plot

EXERCICE N° 3

Scatter Plot — Detection d'anomalies reseau

Niveau
Debutant

Contexte : Detection d'anomalies

Un capteur reseau enregistre pour chaque connexion deux metriques : le volume de donnees envoye (octets) et la duree de la connexion (secondes). La plupart des connexions normales respectent un ratio habituel. Les connexions deviantes peuvent signaler une exfiltration de donnees ou un canal de commande (C2).

Donnees fournies

```
import numpy as np
np.random.seed(42)

# Connexions normales (cluster dense)
duree_normale = np.random.normal(loc=30, scale=8, size=180)
volume_normal = np.random.normal(loc=500, scale=80, size=180)

# Connexions suspectes (outliers)
duree_suspecte = [180, 210, 5, 3, 195, 200]
volume_suspect = [8000, 9500, 8, 12, 7800, 10200]

# Labels : 0 = normal, 1 = suspect
labels = ['normal']*180 + ['suspect']*6
```

Travail demande — Fichier : ex3_scatter_anomalie.py

- 1 Tracer les connexions normales en bleu (s=40, alpha=0.5) et les connexions suspectes en rouge (s=120, alpha=0.9, marker='X').
- 2 Ajouter des annotations pour les 3 points suspects ayant le plus grand volume (indiquer leur coordonnees volume/duree).
- 3 Tracer deux lignes de reference : axvline a duree=100s et axhline a volume=3000 octets (lignes orange en tirets) delimitant la zone suspecte.
- 4 Colorer le fond du quadrant superieur droit (duree > 100 ET volume > 3000) en rouge tres transparent (alpha=0.05) : utiliser ax.axvspan() ou ax.fill_betweenx().
- 5 Titre, labels, legende, sauvegarder sous ex3_scatter_anomalie.png.

RESULTAT ATTENDU

Nuage bleu dense (connexions normales) + 6 croix rouges bien visibles, zone de danger rouge transparente en haut a droite, 3 annotations avec coordonnees, 2 lignes de reference oranges.

BONUS

Calculer la distance euclidienne de chaque point suspect au centroide des connexions normales. Afficher cette distance dans l'annotation.

Exercice 4 Repartition des protocoles — Pie Chart

EXERCICE N° 4

Repartition du trafic — Pie Chart ameliore

Niveau
Debutant

Contexte : Audit de trafic reseau

Un pare-feu a enregistre 24h de trafic reseau. Vous devez produire une representation visuelle claire de la repartition du trafic par protocole applicatif pour le rapport mensuel de securite.

Donnees fournies

```
protocoles = ['HTTPS', 'HTTP', 'DNS', 'SSH', 'FTP', 'SMTP', 'Autre']
volumes_gb = [48.3, 12.1, 5.8, 3.2, 1.9, 2.7, 4.1]
couleurs = ['#2196F3', '#64B5F6', '#4CAF50', '#FF5722', '#FFC107', '#9C27B0', '#78909C']
```

Travail demande — Fichier : ex4_pie_protocoles.py

- 1 Tracer un camembert avec les volumes comme valeurs, les labels de protocoles et les pourcentages affiches (autopct='%1.1f%%').
- 2 Faire ressortir (explode=0.08) le protocole SSH pour attirer l'attention sur le trafic chiffre potentiellement suspect.
- 3 Ajouter un donut effect : tracer un cercle blanc au centre (plt.Circle) pour transformer le pie en donut, et afficher le total en Go au centre.
- 4 Ajouter une legende externe avec la valeur en Go pour chaque protocole.
- 5 Sauvegarder sous ex4_pie_protocoles.png.

Indice — Donut chart

```
fig, ax = plt.subplots(figsize=(8, 8))
wedges, texts, autotexts = ax.pie(
    volumes_gb, labels=protocoles, colors=couleurs,
    autopct='%1.1f%%', startangle=140,
    explode=[0,0,0,0.08,0,0,0]) # SSH en evidence

circle = plt.Circle((0,0), 0.6, color='white') # trou central
ax.add_patch(circle)

total = sum(volumes_gb)
ax.text(0, 0, f'{total:.1f} Go', ha='center', va='center', # texte au centre
        fontsize=16, fontweight='bold', color=BLUE_DARK)
```

RESULTAT ATTENDU

Donut chart colore, SSH ecarte, total en Go affiche au centre, pourcentages lisibles sur chaque section, legende laterale avec valeurs en Go.

BONUS

Ajouter un second graphique a cote (plt.subplots(1,2)) : un bar chart horizontal triant les protocoles par volume decroissant pour faciliter la comparaison precise.

Exercice 5 Distribution des durees de connexion — Histogramme

EXERCICE N° 5

Histogramme — Distribution des durees de session

Niveau
Intermediaire

Contexte : Profil comportemental

Un proxy d'entreprise journalise la duree de chaque session utilisateur. Les sessions tres longues (> 3600s) sont potentiellement associees a des transferts de donnees importants ou a des connexions automatisees (bots, tunnels). Vous devez visualiser et comparer la distribution des durees entre sessions normales et sessions signalees.

Donnees fournies — a generer

```
import numpy as np
np.random.seed(0)

# Sessions normales : duree moyenne 8 min, ecart-type 4 min
sessions_normales = np.random.exponential(scale=480, size=800) # en secondes

# Sessions signalees : durees plus longues ou tres courtes
sessions_signalees = np.concatenate([
    np.random.normal(loc=3800, scale=400, size=60), # longues
    np.random.uniform(low=1, high=15, size=40) # tres courtes (scan ?)
])
```

Travail demande — Fichier : ex5_histo_durees.py

- 1 Créer une figure avec deux sous-graphiques verticaux (fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))).
- 2 Sur ax1 : histogramme des sessions normales (bins=40, couleur steelblue, kde=False) avec Seaborn sns.histplot().
- 3 Sur ax2 : histogramme des sessions signalees (bins=30, couleur coral) avec la courbe KDE (kde=True).
- 4 Sur chaque axe : tracer une ligne verticale (axvline) a la mediane et a la moyenne, avec annotations legendes.
- 5 Utiliser ax.set_xscale('log') sur ax1 pour mieux visualiser la distribution exponentielle. Titres distincts sur chaque sous-graphique.
- 6 plt.tight_layout() et sauvegarder sous ex5_histo_durees.png.

RESULTAT ATTENDU

Deux histogrammes empiles. Haut : sessions normales avec echelle log sur X. Bas : sessions signalees avec courbe KDE, pics a gauche (scans courts) et a droite (longues sessions), lignes mediane/moyenne.

BONUS

Superposer les deux distributions sur un meme axe (`ax.hist(..., alpha=0.5)` pour les deux) et ajouter un test statistique simple (afficher la moyenne et l'ecart-type de chaque groupe dans le titre).

Exercice 6 Correlation des metriques reseau — Heatmap

**EXERCICE
N° 6**

Heatmap — Matrice de correlation des metriques reseau

Niveau
Intermediaire

Contexte : Analyse multivariee

Un collecteur SNMP enregistre 6 metriques reseau par heure sur 30 jours. Avant d'entrainer un modele de detection d'anomalies, l'analyste doit identifier les metriques correlees (redondantes) et les metriques independantes (informatives). La heatmap de correlation est l'outil de reference pour cette analyse exploratoire.

Donnees fournies — a generer

```
import numpy as np
import pandas as pd
np.random.seed(7)

n = 720 # 30 jours x 24 heures
base = np.random.normal(100, 15, n)

df = pd.DataFrame({
    'bande_passante_Mbps' : base + np.random.normal(0, 5, n),
    'nb_connexions'       : base * 0.8 + np.random.normal(20, 8, n),
    'latence_ms'          : 100 - base * 0.3 + np.random.normal(0, 10, n),
    'paquets_perdus_pct'  : np.random.exponential(0.5, n),
    'cpu_firewall_pct'     : base * 0.5 + np.random.normal(30, 6, n),
    'nb_alertes_ids'      : np.random.poisson(3, n).astype(float)
})
```

Travail demande — Fichier : ex6_heatmap_correlation.py

- 1 Calculer la matrice de correlation avec `df.corr()` et l'arrondir a 2 decimales.
- 2 Creer une heatmap Seaborn (`sns.heatmap`) avec `annot=True`, `fmt='.2f'`, `cmap='coolwarm'`, `vmin=-1`, `vmax=1`, `linewidths=0.5`.
- 3 Masquer le triangle inferieur pour ne garder que le triangle superieur (`mask=np.tril(np.ones_like(corr, dtype=bool))`).
- 4 Pivoter les labels de l'axe X de 45 degres et ajuster leur taille pour la lisibilite.
- 5 Ajouter un titre explicatif et sauvegarder sous `ex6_heatmap_correlation.png`.

RESULTAT
ATTENDU

Triangle superieur de la matrice 6x6, couleur rouge pour correlations positives fortes (ex: bande_passante/nb_connexions), bleue pour correlations negatives (ex: bande_passante/latence), blanc pour correlations nulles. Valeurs numeriques lisibles dans chaque case.

BONUS

Identifier programmatiquement les paires de metriques dont la correlation absolue est > 0.7 et les afficher dans la console sous forme de tableau. Ajouter un encadre dans le graphique avec ces paires identifiees.

Exercice 7 Distribution par groupe — Violin + Boxplot

EXERCICE
N° 7

Violin & Boxplot — Comparaison par protocole

Niveau
Intermediaire

Contexte : Analyse comparative

L'equipe reseau souhaite comparer la distribution des tailles de paquets selon 4 protocoles differents (HTTP, HTTPS, SSH, DNS) pour detecter des comportements anormaux. Par exemple, un grand nombre de paquets DNS tres volumineux peut signaler un tunnel DNS.

Donnees fournies — a generer

```
import numpy as np
import pandas as pd
np.random.seed(12)

# Tailles de paquets simulees (octets) par protocole
data = {
    'HTTP' : np.random.normal(loc=800, scale=200, size=300).clip(60, 1500),
    'HTTPS' : np.random.normal(loc=950, scale=180, size=300).clip(60, 1500),
    'SSH' : np.concatenate([
        np.random.normal(60, 10, 250), # paquets de controle
        np.random.normal(900, 150, 50) # paquets de donnees
    ]),
    'DNS' : np.concatenate([
        np.random.normal(80, 20, 260), # requetes normales
        np.random.normal(600, 100, 40) # DNS suspects (tunnel ?)
    ])
}
df = pd.DataFrame(data).melt(var_name='protocole', value_name='taille_octets')
```

Travail demande — Fichier : ex7_violin_boxplot.py

1

Creer une figure avec deux sous-graphiques cote a cote : ax1 pour le violin plot, ax2 pour le boxplot.

2

Sur ax1 : `sns.violinplot(data=df, x='protocole', y='taille_octets')` avec `palette='Set2'`. Le violin plot visualise la forme complete de la distribution.

3

Sur ax2 : `sns.boxplot(data=df, x='protocole', y='taille_octets')` avec `palette='Set2'`. Superposer les points individuels avec `sns.stripplot()` (`alpha=0.3`, `dodge=True`, `jitter=True`).

4

Ajouter une ligne horizontale de reference a `y=400` octets (seuil entre petits et grands paquets) sur les deux axes (`axhline`, couleur rouge en tirets).

5

Titres distincts pour chaque sous-graphique, `fig.suptitle()` pour le titre global, `plt.tight_layout()`, sauvegarder sous `ex7_violin_boxplot.png`.

RESULTAT
ATTENDU

Deux graphiques cote a cote. Gauche : violins montrant la bimodalite de SSH et DNS (deux bosses). Droite : boxplots avec points individuels superposes, outliers visibles, ligne de reference a 400 octets.

BONUS

Calculer et afficher dans le titre de `ax2` le pourcentage de paquets DNS dépassant 400 octets. Mettre ce chiffre en gras rouge si $> 10\%$.

Exercice 8 Mini-projet — Tableau de bord SOC

MINI-PROJET
FINAL

Tableau de bord SOC — Dashboard 4
graphiques

Niveau
Avance

Contexte : Mission finale — Analyste SOC junior

Vous venez d'etre embauche comme analyste SOC junior. Votre responsable vous demande de produire un tableau de bord visuel synthetisant une journee d'activite reseau. Ce dashboard sera affiche sur l'ecran de supervision de l'equipe. Il doit etre lisible d'un coup d'oeil et regrouper les 4 visualisations les plus importantes.

Fichier de sortie unique : `dashboard_soc.png`

Le dashboard doit contenir 4 graphiques disposes en grille 2x2 dans une seule figure (`fig`, axes = `plt.subplots(2, 2, figsize=(16, 10))`).

Position	Graphique	Donnees a représenter	Axe / axes
<code>axes[0,0]</code>	Line plot	Volume de trafic entrant/sortant par heure (24h)	X = heure, Y = Mbps, 2 courbes
<code>axes[0,1]</code>	Bar chart horiz.	Top 8 adresses IP source par nombre de connexions	Y = IP, X = nb connexions
<code>axes[1,0]</code>	Countplot Seaborn	Nombre d'alertes IDS par type sur la journee	X = type alerte, Y = count
<code>axes[1,1]</code>	Scatter plot	Volume vs duree — toutes les connexions (colorier par classe normale/suspecte)	X = duree, Y = volume

Donnees a generer pour le dashboard

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
np.random.seed(99)

# --- Graphique 1 : trafic horaire ---
heures = list(range(24))
entrant = [12, 8, 6, 5, 4, 5, 9, 18, 32, 38, 40, 42, 38, 35, 37, 41, 45, 39, 28, 21, 16, 14, 12, 9]
sortant = [8, 5, 4, 4, 3, 4, 7, 14, 25, 30, 31, 32, 30, 28, 29, 31, 34, 30, 22, 17, 13, 11, 9, 7]

# --- Graphique 2 : top IP ---
top_ips = ['192.168.1.45', '10.0.0.12', '172.16.0.8', '192.168.2.3',
           '10.0.0.99', '192.168.1.120', '172.16.0.55', '10.0.0.200']
nb_conn = [342, 287, 198, 165, 134, 112, 98, 76]

# --- Graphique 3 : alertes IDS ---
types_ids = (['Port Scan']*45 + ['Brute Force']*32 + ['DDoS']*18 +
             ['SQLi']*12 + ['XSS']*8 + ['C2 Traffic']*5)
df_ids = pd.DataFrame({'type_alerte': types_ids})

# --- Graphique 4 : scatter volume/duree ---
durees_n = np.random.normal(30, 8, 200)
volumes_n = np.random.normal(500, 80, 200)
durees_s = [180, 200, 4, 2, 190, 195, 3]
volumes_s = [8500, 9200, 9, 7, 7900, 8800, 12]
classes = ['Normal']*200 + ['Suspect']*7
```

Consignes de mise en forme du dashboard

- 1 Utiliser `sns.set_theme(style='whitegrid', font_scale=0.9)` en debut de script pour unifier l'apparence.
- 2 Chaque sous-graphique doit avoir un titre concis et informatif (`ax.set_title()`) et des labels d'axes clairs.
- 3 Le graphique 2 (top IP) doit etre trie par ordre decroissant. Les 3 premieres IP doivent etre colorees en rouge (risque eleve).
- 4 `fig.suptitle()` : titre global 'Dashboard SOC — 24h de surveillance reseau', fontsize=16, fontweight='bold'.
- 5 `plt.tight_layout(rect=[0, 0, 1, 0.96])` pour laisser de la place au titre global.
- 6 Sauvegarder sous `dashboard_soc.png` (dpi=200, bbox_inches='tight').

RESULTAT ATTENDU

Figure 16x10 pouces en 4 quadrants : haut gauche = courbe trafic double, haut droit = barres horizontales IP trie'es, bas gauche = countplot alertes, bas droit = scatter normal/suspect. Titre global en haut. Apparence uniforme Seaborn.

BONUS

Ajouter un 5e graphique en bas en pleine largeur (`axes[2,0]` avec `colspan=2` via `gridspec`) : une heatmap montrant l'activite par heure et par type d'alerte (colonne = heure 0-23, ligne = type d'alerte, valeur = nombre).